

AN AUDITABLE LEAN 4 SCAFFOLD FOR BEAL’S CONJECTURE: A CONDITIONAL REDUCTION AND ITS LEVEL-26 COMPUTATIONAL EVIDENCE BASE

DAVID FOX

ABSTRACT. We present two companion Lean 4 developments toward Beal’s conjecture ($A^x + B^y = C^z$ with $\min(x, y, z) > 2$ and $\gcd(A, B, C) = 1$ implying a contradiction) and describe the auditable scaffold architecture that organizes both. The first development, **beal-conjecture**, reduces the conjecture to exactly five explicit, ordinary theorem hypotheses via a single assembly theorem; none of the five is a global **axiom** declaration, and the reduction is machine-checked end to end. The second, **beal-level-26-foundations**, is a companion computable evidence base at level 26: PARI/GP-verified two-descent, Selmer computations, and cross-checks against LMFDB for the curves *26a1* and *26b1*, all reproducible from committed, hash-locked certificate data. We do *not* claim a proof of Beal’s conjecture. Our contribution is methodological as much as mathematical: a uniform Core/Wrapper/named- hypothesis layering with automated axiom-footprint auditing (**#print axioms** on every declaration, enforced in continuous integration), and an explicit translation table separating genuinely axiom-free numerical bookkeeping from open mathematical obligations, so that no internal label can be misread as more than it is. Every claim in this note is anchored to a specific, versioned, DOI-archived release; we point to those records rather than re-deriving their contents inline.

CONTENTS

1. Introduction	1
1.1. Two repositories, one method	2
1.2. What is, and is not, claimed	2
2. The Conjecture and the Conditional Reduction	2
3. The Audit Architecture	3
3.1. Three layers	3
3.2. Tooling	4
3.3. A necessary disclaimer: “none” is a bookkeeping label, not a proof of the conjecture	4
4. Computational Evidence Base at Level 26	4
5. A Zero-Axiom Corollary: Beal Implies Fermat	5
6. Data and Reproducibility	5
7. Remaining Obligations and Future Work	6
Acknowledgments	6
References	6

2020 *Mathematics Subject Classification.* 11D41, 11G05, 11F11, 68V20.

Key words and phrases. Beal conjecture, Frey curve, $X_0(26)$, Lean 4, interactive theorem proving, level lowering, axiom auditing, reproducibility.

1. INTRODUCTION

Beal’s conjecture, advanced by Andrew Beal and first published as a prize problem in 1997 [1], asserts that for positive integers A, B, C, x, y, z with $x, y, z \geq 3$,

$$A^x + B^y = C^z \implies \gcd(A, B, C) > 1.$$

No proof is known, and this note does not offer one. What we offer is an *auditable* pair of Lean 4 developments that make the logical structure of the standard elliptic-curve approach to Beal completely explicit, and a description of the audit methodology that keeps that structure honest as the developments grow.

1.1. Two repositories, one method. The work is split across two repositories that play different roles.

- `beal-conjecture` carries the *conditional assembly*: a single theorem, `Beal.Final.ConditionalBealTheorem`, that derives the unrestricted conjecture from five named mathematical premises, each an ordinary function argument rather than a global axiom.
- `beal-level-26-foundations` carries the *computable evidence base*: explicit, reproducible, PARI/GP- and LMFDB-cross-checked computation at level 26 (the curves 26a1, 26b1, their two-descent, and the associated Selmer data), together with a Galois-representation scaffold that narrows the shape of the remaining Tate/Ribet/modularity/Taylor–Wiles obligations without discharging them.

Both repositories are organized under the same audit discipline, which we describe in Section 3 and regard as the primary methodological contribution of this note. Detailed mathematical content — discriminants, certificate hashes, exact Lean identifiers, and CI logs — is not reproduced at length here; instead we cite the specific archived Zenodo record and repository tag that carries each claim, so a reader can verify the primary source directly (Section 6).

1.2. What is, and is not, claimed. To avoid any ambiguity for a reader who does not open the Lean source: the unrestricted statement of Beal’s conjecture is not inhabited in either repository. Section 2 states exactly what `ConditionalBealTheorem` establishes (a genuine but conditional result). Section 3 states exactly what the level-26 repository’s internal “none”-labeled declarations establish, and explicitly separates that from the conjecture itself, because the two are easy to conflate from repository README prose alone.

2. THE CONJECTURE AND THE CONDITIONAL REDUCTION

Definition 2.1. A Beal solution is a tuple $(A, B, C, x, y, z) \in \mathbf{Z}_{>0}^6$ with $x, y, z > 2$, $A^x + B^y = C^z$, and $\gcd(A, \gcd(B, C)) = 1$.

Definition 2.2 (`BealConjecture`, `lean/Beal/B01.Def.lean`). No Beal solution exists.

This is the literal, unrestricted statement: a universally quantified proposition over all positive integers, with no numerical restriction. We verified this directly against the current repository source rather than assuming it from documentation.

The strategy for approaching Definition 2.2 via elliptic curves originates with Frey’s observation [2] that a hypothetical primitive solution carries a geometric structure incompatible with modularity. `beal-conjecture` v11.0.0 assembles this

strategy along a level-26 route through $X_0(26)$ and its Jacobian $J_0(26)$, culminating in a single theorem.

Theorem 2.3 (`Beal.Final.ConditionalBealTheorem`). *Given*

- (1) *`J0DecompositionSoundness_26` — the soundness of the displayed decomposition of $J_0(26)$ used to locate the rational points of $X_0(26)$;*
- (2) *`MwrankCertificateSoundness_26` — the soundness of the supplied `mwrnk` rank certificate for the relevant Mordell–Weil group;*
- (3) *`FormalImmersionSoundness_26` — the soundness of the formal-immersion criterion identifying the finite mod-3 matrix with the actual cotangent map at the boundary point;*
- (4) *`FreyCurveExists` — existence of the Frey curve construction for a hypothetical primitive Beal solution; and*
- (5) *`LevelLowering_26` — the modularity and Ribet level-lowering data needed once that Frey curve is supplied,*

`BealConjecture` holds.

Each of the five premises in Theorem 2.3 is stated as an ordinary hypothesis of the theorem — not a `axiom` declaration — so the dependency is visible in the type signature and cannot silently propagate through unrelated declarations. The proof term constructs a noncuspidal rational point on $X_0(26)$ from a hypothetical primitive solution and contradicts the four-cusp characterization supplied by premises (1)–(3); we do not reproduce the twelve-line proof term here, as it is short, self-contained, and directly readable in the cited release (Table 1).

Remark 2.4. *Premise (3) is the geometric compatibility gap: the finite mod-3 matrix used in the formal-immersion criterion is derived from an explicit basis change on normalized eigenform coefficients, but no theorem in the repository identifies it with the literal Abel–Jacobi cotangent map. Premise (2)’s Selmer cardinality is proved only for an explicitly supplied carrier group; its cohomological identification with the true Selmer group of the curve remains external. These characterizations are taken verbatim from the repository’s own citation metadata, which we regard as the authoritative, versioned statement of what remains open.*

3. THE AUDIT ARCHITECTURE

The methodological contribution of this note is the discipline that keeps a large, multi-hundred-declaration Lean development honest about exactly which classical axioms, and which named mathematical hypotheses, any given result depends on. We describe the three-layer pattern used across both repositories and the tooling that enforces it.

3.1. Three layers.

- **Core.** Files suffixed `_Core.lean` carry zero imports from Mathlib and zero axioms beyond `propext`. Every Core declaration’s type is the mathematical claim itself; every Core proof is verified with `#print axioms` to return `[]` or `[propext]`.
- **Wrapper.** Ordinary (non-`_Core`) files connect Core statements to Mathlib types (reals, `GCDMonoid`, field norms). The permitted axiom budget here is exactly `{propext, Classical.choice, Quot.sound}` — Mathlib’s own classical footprint, never a project-introduced `sorry` or ad hoc axiom.

- **Named hypothesis.** A small number of declarations — Theorem 2.3’s five premises, and `beal-conjecture`’s historical `modularity_hypothesis` — are declared exactly once as the boundary of current mathematical knowledge, and every downstream use is required (and CI-enforced) to route through that single named declaration rather than reintroducing the assumption informally elsewhere.

3.2. **Tooling.** Both repositories ship executable audits, run on every push:

- a literal `grep -r "sorry"` over the Lean tree, required to return empty;
- `#print axioms` on every Core declaration, checked against the permitted set above;
- a boundary check that a named hypothesis (e.g. the level-26 repository’s five-premise theorem, or `modularity_hypothesis` in earlier drafts) is not accidentally depended upon by a declaration that claims to be axiom-free.

3.3. **A necessary disclaimer: “none” is a bookkeeping label, not a proof of the conjecture.** `beal-level-26-foundations` labels declarations `none` when their `#print axioms` output is empty (beyond the kernel), and `propext` only when it is exactly `{propext}`. These labels describe axiom footprint, and nothing else. In particular:

- The repository’s `BealForall` is a `structure` wrapping a single field, `frey_beal_forall_none_formula`, which itself unfolds to a conjunction of numeral identities and internal token equalities (for instance $2 \cdot 13 = 26$, and the recorded discriminants of $26a1$ and $26b1$), each decidable by `rfl`. Declarations such as `BealForall_real_witness_none` and `beal_forall_in_kernel_from_beal_forall_none_separated` are proofs of *this* structure — a build-time consistency checklist — not of the quantified Beal statement.
- The quantified statement itself,

$\forall A B C m n p, 2 < m \rightarrow 2 < n \rightarrow 2 < p \rightarrow A^m + B^n = C^p \rightarrow \gcd(A, \gcd(B, C)) > 1$,
 named `beal_forall_from_Is13Case_sketch` in the repository, is proved *uninhabited* in the ordinary sense of having no known term, and the repository carries an explicit lock, `original_beal_forall_sketch_type.eq`, pinning down that this is the correct, unrestricted type.

We flag this explicitly because repository-level release notes describe some of these declarations with language such as “unconditional” and “BOTH none,” which is accurate only in the narrow bookkeeping sense above; read without this section, it is easy to mistake for a claim about Definition 2.2. No such claim is made, here or in the source.

4. COMPUTATIONAL EVIDENCE BASE AT LEVEL 26

Independently of the bookkeeping discussed in Section 3.3, `beal-level-26-foundations` contains genuine, reproducible computational number theory that narrows the shape of Theorem 2.3’s premises at level 26:

- Two-descent on the elliptic curves $26a1$ ($\Delta = -17576$) and $26b1$ ($\Delta = -1664$), computed via `cypari2`/PARI-GP, with the resulting Selmer, rank, and torsion data committed as SHA-256-hash-locked certificate JSON and cross-checked against LMFDB display data.

- An explicit finite scaffold recording the conductor arithmetic, cyclotomic character, and unramified/semistable/finite-flat conditions attached to the residual representation at the prime 13, together with an explicit infinite family of auxiliary primes $q \equiv 1 \pmod{13^n}$ used in the Taylor–Wiles patching argument’s standard template.

We emphasize what this evidence does *not* do: it does not discharge any of the five premises of Theorem 2.3, and the Galois-representation and Hecke-algebra structures referenced in the scaffold are typed placeholders recording the *shape* of the objects Tate’s algorithm, Ribet’s theorem, and Taylor–Wiles patching would need to supply — not verified instances of those theorems. We point to the specific, versioned records (Table 1) rather than reproducing certificate contents inline; every certificate is reproducible from the committed script and PARI-GP version recorded in that release.

5. A ZERO-AXIOM COROLLARY: BEAL IMPLIES FERMAT

Independently of the conditional reduction, the following is a direct, zero-axiom consequence of Definition 2.2 at equal exponents, verified against the current repository source.

Corollary 5.1 (`beal_implies_fermat21_core`). *BealConjecture21Core $\implies$ FermatLastTheorem21Core: if no primitive Beal solution exists, then for all positive integers a, b, c and all $n > 2$, $a^n + b^n \neq c^n$ whenever $\gcd(a, \gcd(b, c)) = 1$.*

Proof. A Fermat instance at exponent $n > 2$ with coprime a, b, c is, by definition, a Beal solution with $x = y = z = n$; the corollary is the one-line term `fun h a b c n _ hSol => h a b c n hSol`, checked with `#print axioms` to depend on nothing beyond the kernel. $\square$

6. DATA AND REPRODUCIBILITY

Table 1 anchors every claim above to a specific, archived, versioned record. We follow the convention, used throughout both repositories, of minting a Zenodo DOI on every tagged GitHub release and recording it in that release’s own documentation before any later release references it.

TABLE 1. Key repositories, tags, and DOI records.

Repository / tag	DOI	Content
<code>beal-conjecture</code> (concept)	10.5281/zenodo.2 2041831	Concept DOI; resolves to latest version.
<code>beal-conjecture</code> <code>v11.0.0</code>	10.5281/zenodo.2 2281075	<code>ConditionalBealTheorem</code> , five named premises (Section 2).
<code>beal-level-26-</code> <code>foundations</code> (concept)	10.5281/zenodo.2 2272382	Concept DOI; listed as <code>isSupplementTo</code> the conjecture repository above.
<code>beal-level-26-</code> <code>foundations</code> <code>v7.1.0</code>	10.5281/zenodo.2 2632209	Level-26 evidence base and audit-labeled bookkeeping (Sections 3–4).
<code>beal-level-26-</code> <code>foundations</code> <code>v7.1.1</code>	10.5281/zenodo.2 2635221	Documentation catch-up recording the <code>v7.1.0</code> mint.

Both repositories are MIT-licensed and publicly readable; every audit script referenced in Section 3 is runnable from a fresh clone with `lake build` followed by the

repository’s `verify-scaffold.sh` (level-26 repository) or `scripts/verify-all.sh` (conjecture repository).

7. REMAINING OBLIGATIONS AND FUTURE WORK

The path from the present state to a genuine unconditional result runs through discharging Theorem 2.3’s five premises in full generality — most substantially, a Lean formalization of Tate’s algorithm [7] sufficient to compute conductors for the general Frey curve family, a formalization of Ribet’s level-lowering theorem [6] building on Mazur’s irreducibility criterion [5], and the modularity input in the generality Wiles and Taylor–Wiles established [9, 8]. None of these exist in Mathlib at present; the Imperial College London FLT project [4] is formalizing closely related material for Fermat’s Last Theorem, and we regard that effort as the most direct template for discharging the level-lowering premise here. The level-26 evidence base’s finite-flat, unramified, and Taylor–Wiles-family scaffolding (Section 4) is intended to narrow, incrementally, the precise shape those formalizations would need to take at level 26.

ACKNOWLEDGMENTS

The author thanks the Lean 4 and Mathlib communities for the infrastructure that makes this level of audit possible, and the Imperial College London FLT project for the formalization template and for making the shape of the level-lowering gap explicit. The author maintains the *Opera Numerorum* formalization program and its accompanying verification and reproducibility tooling, including the API infrastructure at `zerobeacon.ai`.

REFERENCES

- [1] D. Andrew Beal, *The Beal conjecture and prize*, Notices Amer. Math. Soc. **44** (1997), no. 11, 1436–1437.
- [2] Gerhard Frey, *Links between stable elliptic curves and certain Diophantine equations*, Ann. Univ. Sarav. Ser. Math. **1** (1986), no. 1, 1–40.
- [3] Enrique González-Jiménez and Joan-Carles Lario, *Rational points on twists of $X_0(N)$* , Acta Arith. **143** (2010), no. 2, 149–171.
- [4] Kevin Buzzard et al., *Fermat’s Last Theorem in Lean*, Imperial College London, <https://github.com/ImperialCollegeLondon/FLT>, 2024.
- [5] Barry Mazur, *Rational isogenies of prime degree*, Invent. Math. **44** (1978), no. 2, 129–162.
- [6] Kenneth A. Ribet, *On modular representations of $\text{Gal}(\overline{\mathbf{Q}}/\mathbf{Q})$ arising from modular forms*, Invent. Math. **100** (1990), no. 2, 431–476.
- [7] John Tate, *Algorithm for finding the type of a singular fiber in an elliptic pencil*, in: Modular Functions of One Variable IV, Lecture Notes in Math. 476, Springer, Berlin, 1975, pp. 33–52.
- [8] Richard Taylor and Andrew Wiles, *Ring-theoretic properties of certain Hecke algebras*, Ann. of Math. (2) **141** (1995), no. 3, 553–572.
- [9] Andrew Wiles, *Modular elliptic curves and Fermat’s last theorem*, Ann. of Math. (2) **141** (1995), no. 3, 443–551.
- [10] The Mathlib Community, *The Lean mathematical library*, in: Proc. 9th ACM SIGPLAN Intl. Conf. Certified Programs and Proofs (CPP 2020), ACM, New York, 2020, pp. 367–381.

OPERA NUMERORUM

Email address: `david@zerobeacon.ai`